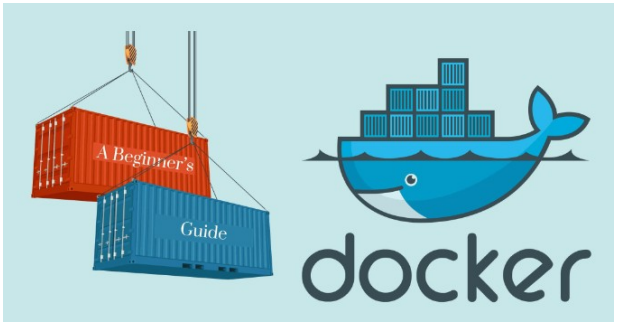


Administración de Sistemas Operativos

UD – 2

Docker

Contenidos





ÍNDICE

1. Virtualización....3
¿Qué es una máquina Virtual ?....3
Contenedores....4
Contenedores para desarrollo y despliegue de aplicaciones....5
Contenedores para despliegue de servicios....5
Ventajas e inconvenientes del uso de contenedores....5
¿Cuándo es adecuado usar contenedores?....6
¿Cómo funcionan los contenedores modernos en Linux?....6
CONTENEDORES DOCKER....7
¿Qué es Docker?....7
La arquitectura de Docker....7
Instalación de Docker....8
Instalación desde el repositorio de Docker-CE (Recomendado)....8
Post instalación....9
Desinstalando Docker en Ubuntu....10
EjercicioS....11
Ejercicio 1....11
Ejercicio 2....11

1. VIRTUALIZACIÓN

La virtualización es un conjunto de tecnologías de hardware y software que permiten la abstracción de hardware, creando así la "ilusión" de administrar recursos virtuales como si fueran recursos reales, de forma transparente para los usuarios. La virtualización es muy utilizada para el despliegue de sistemas, desarrollo de software, análisis de malware, escalado horizontal, etc. Ya que es relativamente sencilla de implementar y puede ahorrar significativamente costes (consumo de energía, mantenimiento, etc.)

1.1 ¿Qué es una máquina Virtual ?

A veces, necesitamos probar un nuevo sistema operativo, una determinada configuración, probar a desplegar un software, etc. pero no está disponible para ello una máquina real. La creación de una máquina virtual utilizando técnicas de virtualización es la solución a este problema. De este modo, una máquina virtual permite simular una máquina (con su sistema operativo) y ejecutar programas como si estuvieran utilizando una máquina real e independiente.

Para la creación de máquinas virtuales generalmente existen varios tipos de tecnologías:

- Máquinas virtuales de proceso.
- Emuladores.
- Hipervisores.
- Contenedores → Docker se engloba en esta categoría.

¿Qué es una máquina virtual de proceso?

Las máquinas virtuales de proceso, son un tipo de máquinas virtuales que permiten ejecutar un programa diseñado para un sistema operativo/arquitectura concreta (distinta de la máquina actual), como un proceso más de nuestra máquina actual. Esto se consigue implementando una máquina virtual de proceso que emula la arquitectura necesaria. Teóricamente, podremos lanzar nuestro programa en cualquier sistema que tenga la máquina virtual de proceso implementada.

Algunos de los principales ejemplos de este tipo de virtualización son:

- **Máquina virtual de Java (JVM)**: ejecuta los bytecodes de Java en cualquier sistema y arquitectura que la tenga implementada.
- **Plataforma .NET:** análoga a la máquina virtual de Java con productos Microsoft.

¿Qué es un emulador?

Un emulador es un software encargado de emular un hardware completo muy específico (por ejemplo, emuladores de videoconsolas antiguas) o una API concreta (por ejemplo, Wine, un software que emula la API de sistemas Windows en otros sistemas operativos, permitiendo ejecutar programas diseñados para Windows en ellos).

¿Qué es un hipervisor?

Un hipervisor es una máquina virtual que simula total o parcialmente un hardware de una máquina, permitiendo la instalación de distintos sistemas operativos (por ejemplo, virtualizar un sistema Windows 11 en una máquina real Linux). Algunos software conocidos que implementan un

hipervisor son: Virtualbox, VMWare, emuladores de consolas, etc. Para saber más: https://es.wikipedia.org/wiki/Hipervisor

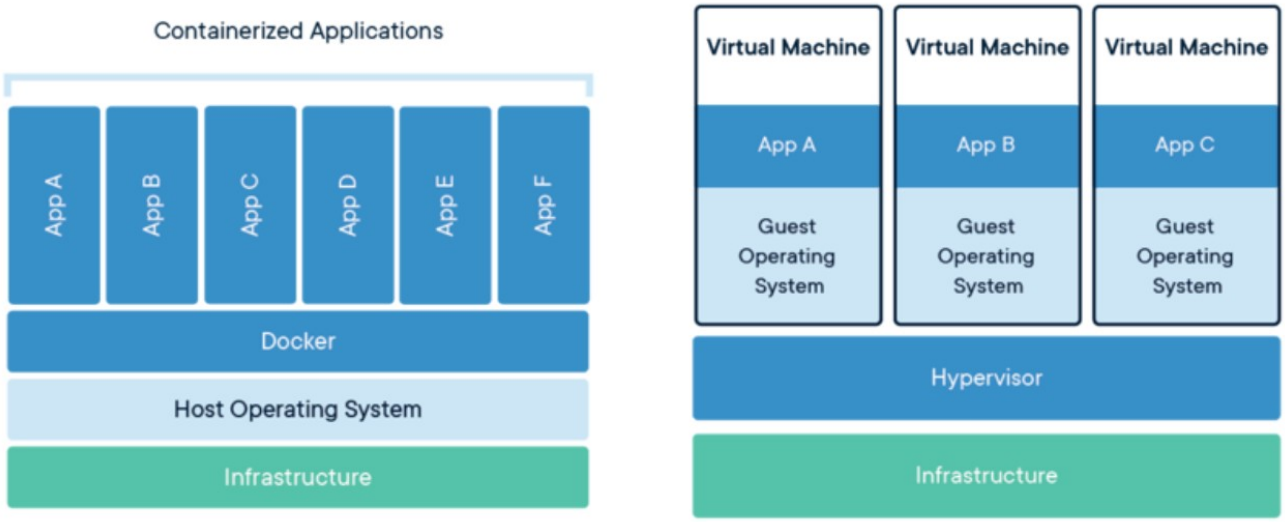
2. CONTENEDORES

Los contenedores son una tecnología de virtualización, que al contrario que un hipervisor (que trata de emular un sistema completo), utiliza el sistema base de la máquina anfitrión y actúa realmente como un "entorno privado" que comparte recursos con el sistema anfitrión, sin virtualizar el hardware completo. En concreto, los contenedores suelen tener entornos privados aislados a nivel de procesos, memoria, sistema de ficheros y red.

Técnicamente, los contenedores son un tipo de virtualización englobada en lo que se llama "**OS Level virtualization**". Para saber más:

https://en.wikipedia.org/wiki/OS-level_virtualization

La siguiente imagen puede ayudarnos a entender el concepto de contenedor.



A la derecha observamos el funcionamiento de un hipervisor, encargado de virtualizar el hardware y donde cada máquina virtual tiene su propio sistema operativo. A la izquierda, observamos un sistema de contenedores, donde no existe esa virtualización del hardware y cada contenedor es un entorno privado.



Para facilitar la comprensión del funcionamiento de los contenedores, vamos a hacer una analogía con los contenedores de transporte marítimo:

- Los contenedores de transporte marítimo, deben cumplir unos estándares (tamaño, peso y forma) para ser transportados.
  - Lo mismo ocurre con los contenedores en virtualización. Mientras cumplan un estándar, pueden ser virtualizados en cualquier máquina que lo soporte (local, servidor, etc.).
- Una vez cumplido el estándar, el tipo de carga del contenedor marítimo es independiente.
  - Lo mismo ocurre con los contenedores en virtualización. Si se cumple el estándar, el software que contenga podrá ser ejecutado sin problemas

2.1 Contenedores para desarrollo y despliegue de aplicaciones

Uno de los principales usos de los contenedores (aunque no el único) es facilitar el desarrollo, distribución y el despliegue de aplicaciones.

- Compilar software es tedioso. Utilizando un contenedor, tenemos el entorno de compilación/depuración montado con las versiones que necesitamos.
- Usar contenedores facilita el testeo, permitiendo la creación de distintos entornos de prueba con diferentes configuraciones, etc.
- Los contenedores nos evitan problemas de compatibilidad al desplegar nuestras aplicaciones, teniendo siempre las versiones adecuadas para ejecutar nuestro software.

2.2 Contenedores para despliegue de servicios

Otro de los principales usos de los contenedores es el despliegue de servidores de distinto tipo (web, correo, bases de datos, DNS, etc.). Además de las ventajas anteriormente citadas de mantener versiones de software, los contenedores nos permiten unificar configuraciones de servidores en local, incluso involucrando a distintos servicios en distintos contenedores, de forma que al desplegarlos en la nube, funcionen exactamente igual que en las pruebas realizadas localmente.

Además, los contenedores facilitan el "escalado horizontal" de servicios, especialmente si se apoyan de herramientas llamadas orquestadores.

Para saber más

https://es.wikipedia.org/wiki/Escalabilidad#Escalabilidad_horizontal

2.3 Ventajas e inconvenientes del uso de contenedores

Algunas de las ventajas del uso de contenedores son:

- Los contenedores ocupan menos espacio, al no tener que replicar en cada uno el sistema operativo que están virtualizando, ya que utilizan el sistema de la máquina anfitrión.
- Al no tener que realizar una virtualización de hardware, la ejecución del software de los contenedores es mucho más rápida, con velocidades cercanas a las nativas.
- Multitud de empresas de software (Microsoft, Apache, Nginx, MySQL, Oracle, Wordpress, Moodle, y un largo etc.) apoyan estas tecnologías y dan soporte tanto incorporando sistemas de contenedores a sus sistemas operativos, como ofreciendo imágenes oficiales de sus productos para que con una sencilla orden, se pueda poner en marcha alguno de sus servicios o aplicaciones.

Algunas de las principales desventajas de los contenedores son:

- Pese a que mejoran enormemente la velocidad respecto a una virtualización por hipervisor, siguen teniendo un rendimiento peor que una ejecución "bare metal" (sobre un sistema real), ya que el aislamiento consume recursos.
- La persistencia y el acceso/modificación a datos persistentes entre contendores es más tedioso que si se realiza sobre una máquina real.
- Los contenedores están pensados generalmente para el uso vía línea de comandos. Aunque técnicamente es posible configurar los contenedores para tener su propio entorno gráfico, este proceso es tedioso.



2.4 ¿Cuándo es adecuado usar contenedores?

El uso de contenedores, suele ser adecuado en los contextos:

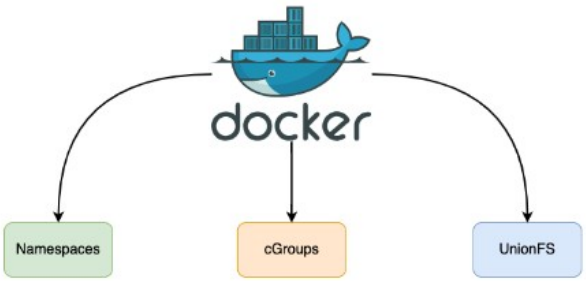
- Como usuarios: queremos probar algo rápido y sin complicarnos mucho en la configuración (por ejemplo, montar un servicio en local para aprender).
  - Para ello, podemos utilizar servicios de distribución de imágenes de contenedores públicas como Docker Hub https://hub.docker.com/
- Como desarrolladores: queremos desarrollar una aplicación que se pueda distribuir en local o desplegar en la nube sin problemas de configuración
  - Podemos usar contenedores, tanto para tener el entorno de desarrollo listo, como para distribuir la aplicación en sí.
- Queremos testear nuestra aplicación con distintas configuraciones, límites de recursos, juegos de prueba, etc.
  - Útil para generar entornos de prueba y despliegue utilizando CI/CD (Continuous Integration/Continuous Delivery) https://es.wikipedia.org/wiki/CI/CD
- Queremos realizar “escalado horizontal” de servicios, es decir, ejecutar múltiples copias de una misma aplicación/conjunto de aplicaciones que funcionan como un clúster.
  - https://es.wikipedia.org/wiki/Escalabilidad#Escalabilidad_horizontal

3. ¿CÓMO FUNCIONAN LOS CONTENEDORES MODERNOS EN LINUX?

Los sistemas más populares de contenedores sobre Linux, han utilizado (entre otras) dos características del kernel aparecidas en versiones relativamente recientes:

- Linux namespaces: permite aislar procesos de forma que vean unos recursos concretos. Los procesos que tienen un “namespace” común pueden ver recursos comunes.
  - Esto, entre otras cosas, nos permite tener procesos “diferentes” de la máquina real a los contenedores, incluso con privilegios diferentes (un proceso puede ser “root” en el contenedor, pero no tiene esos privilegios en la máquina real).
  - Para saber más:
    - https://en.wikipedia.org/wiki/Linux_namespaces
    - https://www.linux.com/news/understanding-and-securing-linux-namespace s/
- **Cgroups**: permite aislar, configurar y limitar el uso de recursos(memoria, procesos, E/S, etc.). Para saber más https://en.wikipedia.org/wiki/Cgroups

En resumen, Linux namespaces nos facilita aislar el sistema y cgroups facilita la limitación/configuración de la disponibilidad de recursos de cada contenedor.



4. CONTENEDORES DOCKER

4.1 ¿Qué es Docker?

Docker es un sistema de contenedores Linux que utiliza las características del núcleo de Linux para permitir el desarrollo y despliegue de aplicaciones. Su web oficial https://www.docker.com/ y su entrada en la wikipedia donde se da información detallada del proyecto https://es.wikipedia.org/wiki/Docker_(software)

Docker es un proyecto de código abierto. Generalmente, dispone de varias versiones:

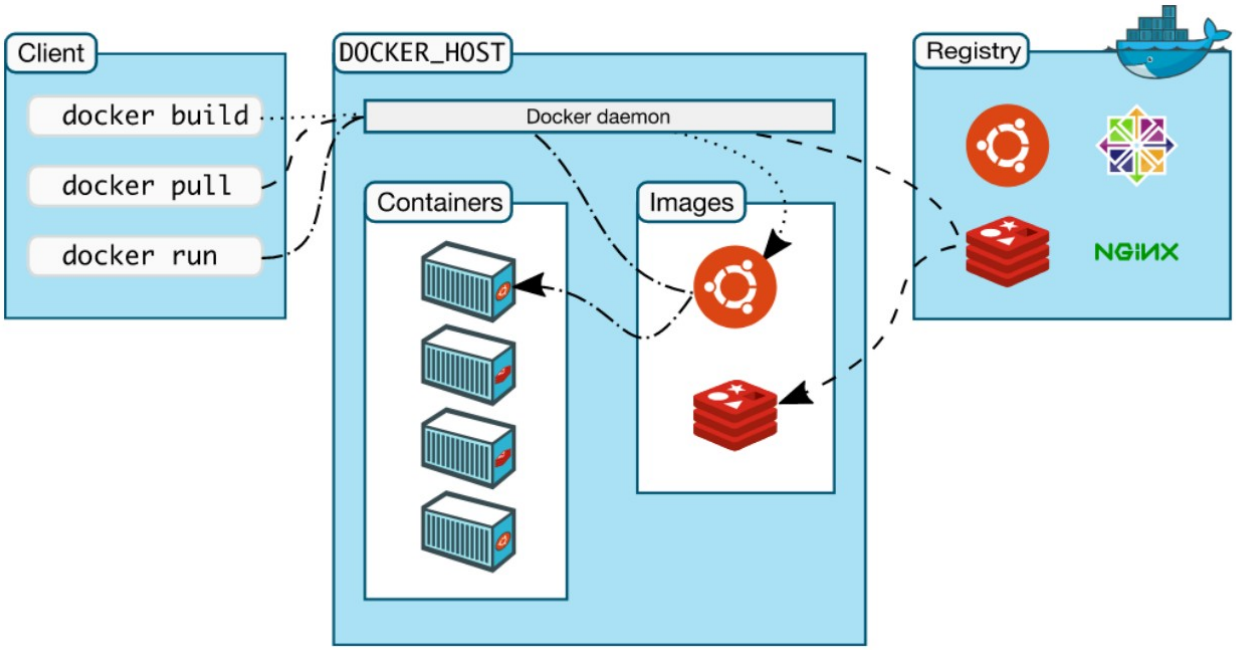
- **Docker CE** (*Community Edition*): el motor de Docker, de código abierto.
- **Docker EE** (*Enterprise Edition*): lo mismo que la versión CE, solo que además incluye certificación de funcionamiento en algunos sistemas concretos y soporte con la empresa Docker Inc.

El sistema de contenedores de Docker es integrable con otros servicios populares en la nube, tales como Google Cloud, Amazon AWS, Microsoft Azure, Digital Ocean y OVH, entre otros.

- **AWS**: https://aws.amazon.com/es/getting-started/hands-on/deploy-docker-containers/
- **Azure**: https://docs.microsoft.com/es-es/learn/modules/run-docker-with-azure-container-instanc es/
- **Google Cloud**: https://cloud.google.com/container-optimized-os?hl=es-419

4.2 La arquitectura de Docker

En esta imagen podemos ver como funciona la arquitectura básica de Docker



5. INSTALACIÓN DE DOCKER

En este apartado vamos a hablar de la instalación de Docker Engine CE (Community Edition) en sistemas Linux en distribuciones basadas en Ubuntu (Ubuntu, KUbuntu, Linux Mint, etc.):

- **Debian** → *https://docs.docker.com/engine/install/debian/*

5.1 Instalación desde el repositorio de Docker-CE (Recomendado)

A continuación detallaremos los pasos para instalar Docker Engine CE en Ubuntu desde el repositorio oficial de Docker CE. Seguiremos los pasos detallados en :

https://docs.docker.com/engine/install/ubuntu/

En primer lugar, deberemos eliminar otras versiones de Docker, por si estuvieran instaladas (por ejemplo, una versión del repositorio oficial de Ubuntu) y pudieran provocar conflictos. Para eliminar las versiones antiguas puede bastarnos con la orden:

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman-docker containerd runc; do sudo apt-get remove $pkg; done
```

Para poner en marcha el repositorio, lo primero que haremos será actualizar nuestro índice de paquetes y tras ello, instalar los paquetes necesarios (si no lo estaban ya) para que se puedan utilizar repositorios con HTTPS.

```
sudo apt-get update
sudo apt-get install ca-certificates curl
```

Una vez realizado este paso, descargamos la clave GPG del repositorio de Docker CE y la incluiremos. Podemos hacer todo con las siguientes línea:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Ahora, solo nos queda añadir el repositorio de Docker CE como fuente para instalación de paquetes.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Para saber si ha funcionado:

```
agustin@agustin-VirtualBox:~/Escritorio$ cat /etc/apt/sources.list.d/docker.list
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu noble stable
```

Ahora si podemos actualizar los repositorios:

```
sudo apt-get update
```

```
agustin@agustin-VirtualBox:~/Escritorio$ sudo apt-get update
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Des:4 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Obj:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [68,0 kB]
Descargados 117 kB en 1s (156 kB/s)
Leyendo lista de paquetes... Hecho
```

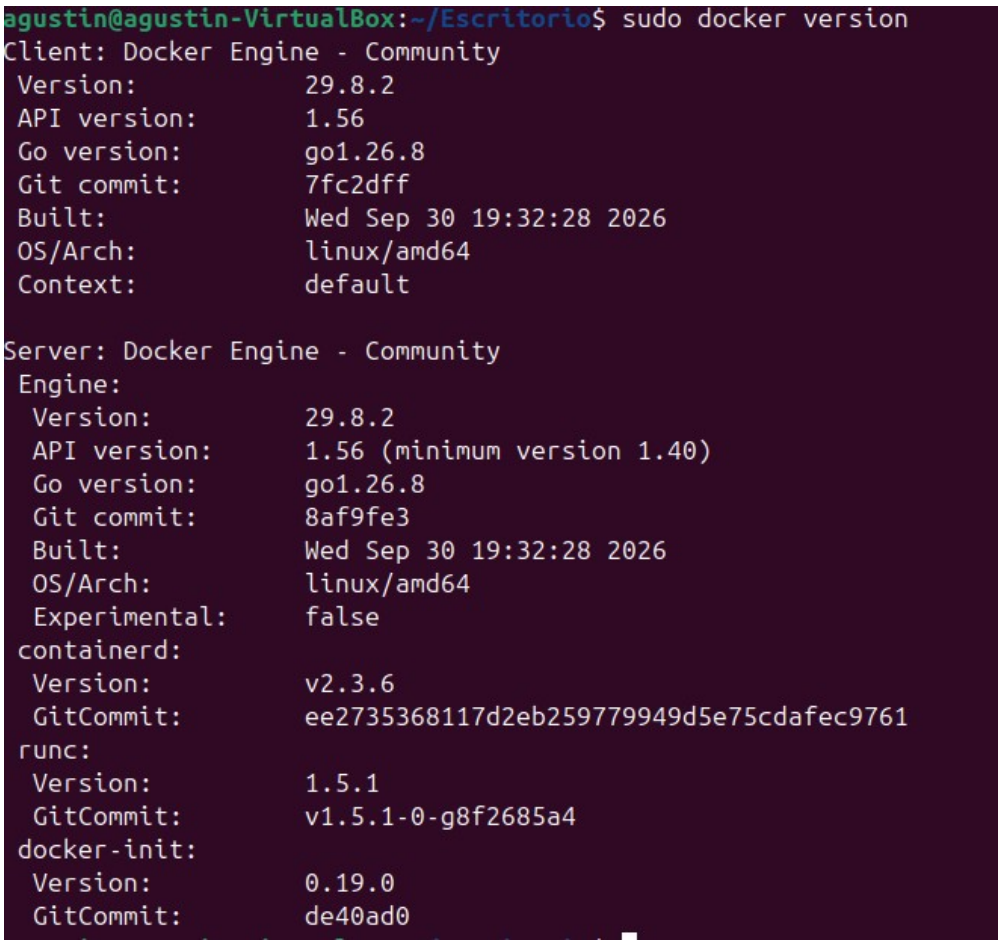
Por último, ya con el repositorio oficial de Docker en nuestro sistema, solo nos queda actualizar el índice de paquetes e instalar la última versión de Docker Engine CE de la siguiente forma:

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Finalmente, comprobaremos que Docker Engine CE se ha instalado correctamente ejecutando:

```
sudo docker version
```

y obteniendo un resultado similar al siguiente:



5.2 Post instalación

En la documentación de Docker, nos proponen algunos pasos de post instalación. Los podéis consultar en

https://docs.docker.com/engine/install/linux-postinstall/

En esta sección vamos a comentar dos de ellos:

- Administrar Docker con usuarios sin privilegios (no root ni sudoers)
- Arrancar Docker desde al inicio.

Permitir administrar Docker con usuarios sin privilegios Docker utiliza sockets Unix. Para la creación y reserva de un socket Unix, es necesario tener permisos de root, por lo cual Docker Engine necesita permisos de root para ejecutarse. A veces, en algunos contextos, puede sernos útil que Docker se ejecute por usuarios sin permisos de root. Pongamos un contexto de un aula de ordenadores, que será utilizada por alumnos con los que queremos realizar actividades que utilizan Docker. Podemos configurar el aula para que dichos alumnos puedan utilizar Docker sin necesidad de proporcionarles una cuenta con permisos de root. Para ello, en primer

lugar crearemos un grupo llamado “docker”.

sudo groupadd docker

Tras ello añadiremos a los usuarios que queremos que usen Docker sin permisos de root al grupo con un comando similar al siguiente, donde $USER es el nombre de usuario:

sudo usermod -aG docker $USER

Para probar que esto funcione, será necesario que el/los usuarios afectados cierren sesión completamente (si la tenían abierta) y la vuelvan a abrir, para que se reevalúe su pertenencia al grupo “docker”. Con esto podrán utilizar comandos Docker sin necesidad de permisos de root.

MUCHO OJO: si lanzaste comandos de Docker usando sudo con uno de estos usuarios antes de esta operación, es posible que al lanzar Docker te aparezca un mensaje similar a este: WARNING:

Error loading config file: /home/user/.docker/config.json
- stat /home/user/.docker/config.json: permission denied

Para arreglar este error, las opciones son eliminar el directorio “.docker” con

sudo rm -rf ~/.docker/

o cambiar el propietario y permisos del directorio, usando:

sudo chown "$USER":"$USER" /home/"$USER"/.docker -R
sudo chmod g+rwx "$HOME/.docker" -R

Activar/desactivar arranque al inicio Para indicar que el servicio de Docker se inicie al arrancar la máquina, podemos indicarlo mediante los siguientes comandos:

sudo systemctl enable docker.service
sudo systemctl enable containerd.service

Si lo que queremos es deshabilitar este arranque automático, podemos usar:

sudo systemctl disable docker.service
sudo systemctl disable containerd.service

Para iniciar/parar/reiniciar los servicios manualmente, podemos usar:

sudo systemctl start/stop/restart docker.service
sudo systemctl start/stop/restart containerd.service

5.3 Desinstalando Docker en Ubuntu

Si en algún momento queremos desinstalar completamente Docker en Ubuntu, podemos usar el siguiente conjunto de comandos. El primer comando eliminará Docker Engine, pero no eliminará contenedores e imágenes presentes en el sistema.

sudo apt-get purge docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin docker-ce-rootless-extras

Si queremos eliminar estos datos, deberemos realizar el borrado manual de las mismas. Aquí el conjunto de comandos para el borrado manual:

sudo rm -rf /var/lib/docker
sudo rm -rf /var/lib/containerd
sudo rm /etc/apt/sources.list.d/docker.list
sudo rm /etc/apt/keyrings/docker.asc

6. EJERCICIOS

6.1 Ejercicio 1

Ejecuta en tu máquina con la instalación completa de Docker el comando:

docker version

Avisa al profesor y enseñale el resultado del comando.

6.2 Ejercicio 2

Ejecuta en tu máquina con la instalación completa de Docker el comando:

docker run hello-world

Avisa al profesor y enseñale el resultado del comando.. El comando mostrará un contenido similar a este:

```
agustin@agustin-VirtualBox:~/Escritorio$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```